```
#include <iostream>
#include <sstream>
#include <vector>
using namespace std;

vector<int> parseIP(string ip) {
    vector<int> a;
    string t;
    stringstream ss(ip);

    while (getline(ss, t, '.')) {
        if (t.empty())
            return {};
        try {
            size_t pos;
            int x = stoi(t, &pos);

            if (pos != t.size())
                return {};

            a.push_back(x);
        } catch (...) {
            return {};
        }
    }
    return a;
}

bool validIP(const vector<int>& ip) {
    if (ip.size() != 4)
        return false;

    for (int x : ip) {
        if (x < 0 || x > 255)
            return false;
    }
    return true;
}

void printIP(const vector<int>& a) {
```

```
    cout << a[0] << "." << a[1] << "."
         << a[2] << "." << a[3];
}

int main() {
    string ipStr;
    int requiredSubnets;

    cout << "Enter IPv4 Address: ";
    cin >> ipStr;

    cout << "Enter Required Number of Subnets: ";
    cin >> requiredSubnets;

    vector<int> ip = parseIP(ipStr);

    if (!validIP(ip) || requiredSubnets <= 0) {
        cout << "Invalid IP address or subnet requirement.\n";
        return 0;
    }

    int first = ip[0];
    int defaultPrefix;

    if (first >= 1 && first <= 126)
        defaultPrefix = 8;
    else if (first >= 128 && first <= 191)
        defaultPrefix = 16;
    else if (first >= 192 && first <= 223)
        defaultPrefix = 24;
    else {
        cout << "Subnetting is allowed only for Class A, B, or C.\n";
        return 0;
    }

    int hostBits = 32 - defaultPrefix;
    int borrowed = 0;

    while ((1LL << borrowed) < requiredSubnets)
        borrowed++;
```

```
if (borrowed > hostBits) {
    cout << "Too many subnets required.\n";
    return 0;
}

int newPrefix = defaultPrefix + borrowed;
int remainingHostBits = 32 - newPrefix;

long long totalSubnets = 1LL << borrowed;
long long totalAddresses = 1LL << remainingHostBits;
long long usableHosts =
    (remainingHostBits >= 2) ? totalAddresses - 2 : 0;

// Calculate default subnet mask
vector<int> defaultMask(4, 0);

for (int i = 0; i < defaultPrefix; i++)
    defaultMask[i / 8] |= (1 << (7 - i % 8));

// Calculate classful network address
vector<int> network = ip;

for (int i = 0; i < 4; i++)
    network[i] &= defaultMask[i];

// Calculate new subnet mask
vector<int> newMask(4, 0);

for (int i = 0; i < newPrefix; i++)
    newMask[i / 8] |= (1 << (7 - i % 8));

cout << "\nDefault Prefix Length: /"
     << defaultPrefix << endl;

cout << "Default Subnet Mask: ";
printIP(defaultMask);

cout << "\nBorrowed Bits: " << borrowed;
cout << "\nNew Prefix Length: /" << newPrefix;
```

```
cout << "\nNew Subnet Mask: ";
printIP(newMask);

cout << "\nTotal Generated Subnets: " << totalSubnets;
cout << "\nRemaining Host Bits: " << remainingHostBits;
cout << "\nTotal Addresses per Subnet: " << totalAddresses;
cout << "\nUsable Hosts per Subnet: " << usableHosts;

cout << "\n\nSubnet Details:\n";

for (long long s = 0; s < totalSubnets; s++) {
    long long offset = s * totalAddresses;
    vector<int> subnet = network;

    for (int i = 3; i >= 0; i--) {
        subnet[i] += offset % 256;
        offset /= 256;
    }

    vector<int> broadcast = subnet;
    long long lastOffset = totalAddresses - 1;

    for (int i = 3; i >= 0; i--) {
        broadcast[i] += lastOffset % 256;
        lastOffset /= 256;
    }

    vector<int> firstHost = subnet;
    vector<int> lastHost = broadcast;

    if (usableHosts > 0) {
        firstHost[3]++;
        lastHost[3]--;
    }

    cout << "\nSubnet " << s + 1 << ":\n";

    cout << "Network Address: ";
    printIP(subnet);
```

```
        cout << "\nFirst Usable Host: ";
        printIP(firstHost);

        cout << "\nLast Usable Host: ";
        printIP(lastHost);

        cout << "\nBroadcast Address: ";
        printIP(broadcast);

        cout << "\nSubnet Mask: ";
        printIP(newMask);

        cout << "\nCIDR Prefix: /" << newPrefix << "\n";
    }

    return 0;
}
```